

CUPID: Leveraging ChatGPT for More Accurate Duplicate Bug Report Detection

Ting Zhang, Ivana Clairine Irsan, Ferdian Thung, David Lo
School of Computing and Information Systems, Singapore Management University, Singapore.

Contributing authors: tingzhang.2019@phdcs.smu.edu.sg;
ivanairsan@smu.edu.sg; ferdianthung@smu.edu.sg; davidlo@smu.edu.sg;

Abstract

Duplicate bug report detection (DBRD) is a long-standing challenge in both academia and industry. Over the past decades, researchers have proposed various approaches to detect duplicate bug reports more accurately. With the recent advancement of deep learning, researchers have also proposed several deep learning-based approaches to address the DBRD task. In the bug repositories with many bug reports, deep learning-based approaches have shown promising performance. However, in the bug repositories with a smaller number of bug reports, i.e., around 10k, the existing deep learning approaches show worse performance than the traditional approaches. Traditional approaches have limitations, too, e.g., they are usually based on the bag-of-words model, which cannot capture the semantics of bug reports. To address these aforementioned challenges, we seek to leverage a state-of-the-art large language model (LLM) to improve the performance of the traditional DBRD approach.

In this paper, we propose an approach called CUPID, which combines the best-performing traditional DBRD approach (i.e., REP) with the state-of-the-art LLM (i.e., ChatGPT). Specifically, we first leverage ChatGPT under the zero-shot setting to identify essential information on bug reports. We then use the essential information as the input of REP to detect duplicate bug reports. We conducted an evaluation by comparing CUPID with three existing approaches on **three datasets**. The experimental results show that CUPID achieves state-of-the-art results, reaching **Recall Rate@10** scores ranging from **0.602 to 0.654** across all the datasets analyzed. In particular, CUPID improves over the prior state-of-the-art approach by **5% - 8%** in terms of **Recall Rate@10** in the datasets. CUPID also surpassed the state-of-the-art deep learning-based DBRD approach by **up to 82%**. Our work highlights the potential of combining LLMs to improve the performance of software engineering tasks.

1 Introduction

As software systems become larger and more complex, they inevitably contain bugs. *Bug reports* are the main channel for users to report bugs to developers. Most software projects use issue tracking systems, such as Bugzilla [1], Jira [2] and GitHub [3], to manage bug reports and track the progress of bug fixing. When users find a bug, they can submit a bug report to the issue tracking system. The developers will then review the bug report and decide whether it is a valid bug report. If the bug report is valid, the developers will work on fixing the bug. However, many bug reports are duplicates of the existing bug reports. For example, in the dataset constructed by Lazar et al. [4], duplicate bug reports represent 12.67% - 23% out of the total bug reports in a system. It is crucial to identify duplicate bug reports as soon as possible to avoid wasting developers' time and effort on fixing the same bug multiple times. To improve the efficiency of bug report management, it is desirable to have an automatic approach to identify duplicate bug reports.

Over the past decades, various **duplicate bug report detection** (DBRD) approaches have been proposed [5, 6]. With the rapid development of deep learning, many deep learning-based approaches have been proposed in recent years [7–9]. They have demonstrated superior performance when the bug repositories are large enough to train the deep learning models. For instance, SABD [7] achieved over 0.6 in terms of Recall Rate@20 in all the experimented datasets. One of the common characteristics of these datasets is that they all contain more than 80k bug reports and over 10k *duplicate* bug reports in the training data, which is large enough to train a deep learning model. It is well acknowledged that deep learning models require a large amount of data to achieve high precision [10]. However, bug repositories in many projects are not large enough to train a deep-learning model.

Based on the dataset provided by Joshi et al. [11], it was discovered that out of the 994 studied GitHub projects that have more than 50 stars and forks, the average number of issues was 2,365. Additionally, it is interesting to note that many active projects, including those with more than 100k stars, have fewer than 10k issues. For example, till 5th May 2024, both ohmyzsh/ohmyzsh [12] and axios/axios [13] have less than 5k issues each, while vuejs/vue [14] has around 10k issues. Therefore, we argue that most projects do not have tens of thousands of issues. The repositories with tens of thousands of issues are considered as *atypical*, while a *typical* repository contains less than or around 10k issues. It is essential to highlight that young and fast-growing projects, although currently having a few issues, require more attention in handling the DBRD challenge. For instance, Significant-Gravitas/Auto-GPT [15], which was initially released on March 30, 2023, now contains less than 3k issues while it gets 162k stars. A recent benchmarking study on DBRD by Zhang et al. [16] also confirms that the performance of deep learning-based approaches loses to information retrieval-based approaches when the bug repositories only contain less or around 10k

duplicate bug reports. How to improve the performance of DBRD in the *typical* bug repositories remains an open problem.

Before the development of deep learning, many non-deep learning-based approaches have been proposed [5, 17–19] (we refer to them as “*traditional* approaches” in this paper). Compared to deep learning-based approaches, these approaches are more promising for detecting duplicate bug reports in typical bug repositories. However, traditional approaches rely on either the vector space model [17] or the bag-of-words model [18]. These models cannot capture the semantics of bug reports. We seek to improve the performance of non-deep learning-based approaches by considering the semantics of bug reports.

Recently, large language models (LLMs), e.g., LLAMA 3 [20], Phi-3 [21, 22], and OPENCHAT [23], have achieved outstanding performance in a multitude of natural language processing (NLP) tasks [24–26]. However, leveraging the potential of LLMs to improve DBRD’s performance is not trivial. The most straightforward way is to directly query LLMs on whether two bug reports are duplicates. However, this is impractical due to the following reasons.

(1) *Time-consuming and costly.* To obtain the potential master bug reports to which a given bug report may be duplicated, we must pair it with all the bug reports available in the repository. When a new bug report is submitted, all previously submitted bug reports are considered duplicate candidates. It is infeasible to query LLMs to compare the given bug report with all the bug reports in the repository, as the LLMs’ response is not instantaneous. While speeding it up is possible (e.g., by running many queries at once), it quickly gets very costly for LLMs such as ChatGPT [27], which operates on a pay-per-use basis for their API usage.

(2) *Ignorance of other bug reports in the repository.* If a method only compares two bug reports at a time, it will not take into account the information present in the other bug reports stored in the repository. Therefore, it would be hard to decide the relative order of all the duplicate candidates to recommend the top- k duplicate candidates. Although one possibility is in addition to querying ChatGPT on whether the bug report pair is duplicated or not, we ask ChatGPT to provide a measure of how confident it is in its answer, expressed as a similarity score or confidence score. However, without considering the information from other bug reports, the similarity score will be less reliable.

To address the challenges mentioned above, we propose a novel approach that combines the advantages of both traditional DBRD approaches and LLMs. We present CUPID, which stands for leveraging ChatGPT for duplicate bug report detection. CUPID aims to tackle the abovementioned challenges when directly querying LLMs for DBRD. We propose to leverage LLMs as an intermediate step to improve the performance of the traditional DBRD approach. Based on the recent benchmarking study by Zhang et al. [16], REP [5] demonstrates the best performance in the datasets with a typical number of issues, which is also the focus of this work. Thus, we select REP as the backbone duplicate retrieval method. Specifically, CUPID leverages state-of-the-art ChatGPT to extract keywords from bug reports and then incorporate them with REP to achieve better performance. By doing so, CUPID avoids using ChatGPT to

compare the given bug report with all the bug reports in the repository. Furthermore, by standing on the shoulder of the traditional DBRD approach, CUPID also takes the information of the other bug reports in the repository into consideration. In particular, $BM25F_{ext}$ used by REP calculates inverse document frequency (IDF), which is a global term-weighting scheme across all the bug reports. Our contribution can be summarized as follows:

- **Approach:** We propose CUPID, which combines modern LLMs with the traditional DBRD technique to enhance the accuracy of DBRD in software systems with the typical number of bug reports.
- **Evaluation:** We evaluate CUPID on **three datasets** from open-source projects and compare CUPID with three prior state-of-the-art approaches. The experimental results indicate that CUPID surpasses the performance of these existing DBRD approaches. Notably, CUPID achieves RR@10 scores ranging from **0.602 to 0.654** across all the datasets analyzed.
- **Direction:** We show that leveraging ChatGPT indirectly in conjunction with existing approaches can be beneficial. We anticipate that this will pave the way for future research to explore innovative ways to utilize state-of-the-art techniques with traditional ones.

The structure of this paper is as follows. Section 2 introduces the background of DBRD. Section 3 presents the details of CUPID. We describe the experimental design in Section 4. Section 5 presents the experimental results. We discuss the threats to validity in Section 6. Finally, Section 8 concludes this paper and discusses future work.

2 Background

In this section, we first introduce the essential concepts, including bug reports and duplicate bug reports, and finally, we discuss the task of DBRD.

Bug reports are the primary means for users to communicate a problem or request features to developers [28]. Software projects usually rely on *issue tracking systems* to collect these bug reports. While the supported fields can vary from system to system, the *textual* information is included in all the issue tracking systems. The *textual* fields in a bug report usually consist of a summary (title) and a description. In Bugzilla or Jira, there are also several *categorical* fields, such as **priority** (bug assignees use this to prioritize their bug), **product**, **component**, etc.

DBRD aims to correctly link a duplicate bug report towards its *master* bug report. Following prior works [5, 7, 17], we denote the first submitted bug report on a specific fault as the *master* bug report and the subsequent bug reports on the same fault as *duplicates*. All the bug reports which are duplicates of each other, including *master* and *duplicates*, are in the same *bucket*. To help understand these concepts easier, we can imagine a *bucket* as a Hash Table, where the key is the master bug report, while the values are duplicate bug reports and themselves. Thus, for a unique bug, both the key and value would be just itself.

In the literature, depending on the problem setting, DBRD has been evaluated in two manners, i.e., (1) *classification* and (2) *ranking*. In the *classification* manner, the

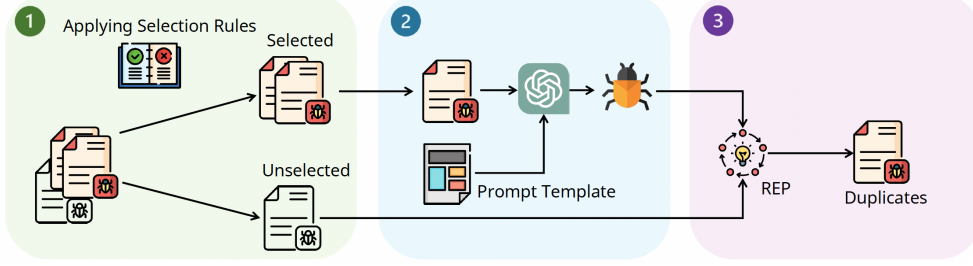


Fig. 1: CUPID contains three stages: In Stage 1, it applies selection rules to select the test bug reports that need to be processed; In Stage 2, it utilizes ChatGPT to process the selected bug reports; In Stage 3, it leverages REP to retrieve potential master bug report for each test bug report.

task is to classify whether two bug reports are duplicates or not. DC-CNN [8] and HINDBR [9] are two recent endeavors in this manner. In the *ranking* manner, the task is to rank the candidate bug reports according to their similarity to the given bug report. Referring to the Hash Table metaphor earlier, given a newly submitted bug report, DBRD technique finds the bucket to which it belongs (also equivalent to linking duplicate bug reports to its master). If it does not belong to any existing bucket, a new bucket in which the key and value are itself should be created. In this work, we adopt the *ranking* manner, which is more practical in real-world applications [7, 16].

In the past decades, researchers have proposed various approaches to address the DBRD task in the ranking manner [29, 30]. Different DBRD approaches mainly differ in (1) feature engineering: which features in bug reports are selected and how these features are represented, and (2) similarity measurement: how to measure the similarity between two bug reports [31]. In terms of *feature engineering*, we further break it down into two parts: (1) what features are selected and (2) how to represent the features. All existing methods use textual information and most of them use categorical information. Textual features, i.e., summary and description, include the most useful information about a bug. Different methods differ in which categorical features to use. To model these features, traditional methods utilize bag-of-words, character-level N-gram, or $BM25_{ext}$ to model textual features [5, 18, 31], while bag-of-words or hand-crafted methods are usually used to model categorical features. Deep learning-based methods utilize word embeddings, such as GloVe [32] and word2vec [33], to represent the textual information. Other types of neural networks, such as HIN2vec [34], are used to represent categorical information. For *similarity measurement*, traditional methods usually use Cosine, Dice, and Jaccard similarity [17, 18]. While some deep learning-based models also adopt this similarity measure [6], some of them leverage neural networks to learn the similarity [7].

3 Approach

We propose CUPID to combine the advantages of both the traditional DBRD approach and LLMs. As mentioned earlier, our work focuses on solving the DBRD challenge in

Table 1: Example content can be matched by the dotted identifier regular expression.

Code snippet:
<code>SELECT * FROM database.schema.table;</code>

Error message:
<code>[ERROR] Failed to execute goal org.apache.maven.plugins:maven-javadoc-plugin: 3.0.1:javadoc (default-cli) on project hadoop-hdfs: An error has occurred in Javadoc report generation: ... omitted</code>

Stack traces:
<code>java.lang.NullPointerException at com.example.MyClass.myMethod(MyClass.java:42) ... omitted</code>

the repositories with a typical number of issues; evidence shows that traditional DBRD approaches would fit more in this condition than deep learning-based approaches [16]. Figure 1 shows an overview of the proposed method. The overall process consists of three main stages: (1) Applying *selection rules* to select the bug reports that need to be processed by ChatGPT, (2) Running ChatGPT with *prompt template* to get the essential keywords of the selected bug reports, and (3) Applying REP to retrieve *potential master bug reports*.

3.1 Applying Selection Rules

Considering the computational cost of ChatGPT, we did not run ChatGPT on all the bug reports in the test dataset. Similarly, in practice, we do not need to run ChatGPT on each newly submitted bug report. To further improve efficiency while keeping accuracy, we explore and propose selection rules. These rules are based on the content of the bug reports to prioritize bug reports that are harder to process by REP while reducing the number of bug reports that are fed into ChatGPT. The selection criterion is based on the content. We select the bug reports whose descriptions contain dotted identifiers or URLs. We use regular expressions `r"\w+\.\w+\.\w{1,}"` and `r"https?:\/\/S+"` to match and select these bug reports. The dotted identifiers usually exist in fully qualified class names, namespaced identifiers, error messages, and stack traces. Table 1 shows examples of the content that can be matched by the dotted identifier regular expression. For developers, this information is useful. However, for a DBRD method, this information can be hard to process. We select these bug reports because not all the code snippets are useful for REP to retrieve the potential master bug reports. We also do not directly remove code snippets. The reason is that we want to keep the original structure of the bug reports so ChatGPT can understand the language better. We then utilize ChatGPT to identify keywords from these bug reports.

3.2 Running ChatGPT with Prompt Template

After Stage 1, we run ChatGPT on the selected bug reports, i.e., the description contains dotted identifiers or URLs. Prompt [35] is a set of instructions that can be used to probe LLMs to generate the target outcome [36]. Prior studies have empirically

Table 2: Query to ChatGPT to Generate Initial Prompt Template and ChatGPT’s Response.

To Get Initial Prompt Template

Provide a concise prompt or template that can be used to identify keywords from the summary and description of a bug report. These keywords will be used as input to detect duplicate bug reports. The output format will be:

Summary: [Selected Keywords]

Description: [Selected Keywords]

Prompt Template

Identify keywords from the summary and description of the bug report that can be used to detect duplicates.

Output format:

Summary: [Selected Keywords]

Description: [Selected Keywords]

Summary: {{Summary}}

Description: {{Description}}

shown that LLMs are sensitive to prompts [37, 38]. Thus, for different tasks, the prompts should be carefully designed to enable LLMs to demonstrate their abilities. To get the most suitable prompt for ChatGPT, we directly query ChatGPT to get the prompt. Table 2 shows the query we used to get the initial prompt and the prompt replied by ChatGPT. The latter one is what we used in CUPID. After getting the response from ChatGPT, we replace the original summary and description in the bug report with the returned identified keywords for summary and description. We keep the remaining part of the bug report unchanged.

3.3 Retrieving Potential Master Bug Reports

Considering the superiority of REP in the task of DBRD shown in a recent study [16], especially on projects with a typical number of issues, we use REP as the DBRD approach in CUPID. Here, we briefly introduce the REP approach to make the paper self-contained. We refer the readers to the original paper [5] for more details.

As shown in Formula 1, REP is a linear combination of seven features, including **textual** features and **categorical** features.

$$\text{REP}(d, q) = \sum_{i=1}^7 w_i \cdot \text{feature}_i \quad (1)$$

, where d is the bug report in the repository R , q , is the query (i.e., new bug report), w_i is the weight of the i -th feature, and feature_i is the i -th feature. The first two features are both **textual** features, and the rest five features are **categorical** features.

Formula 2 shows how to get each feature.

$$\begin{aligned}
\text{feature}_1(d, q) &= \text{BM25F}_{ext}(d, q) \text{ //of unigrams} \\
\text{feature}_2(d, q) &= \text{BM25F}_{ext}(d, q) \text{ //of bigrams} \\
\text{feature}_3(d, q) &= \begin{cases} 1, & \text{if } d \cdot \text{prod} = q \cdot \text{prod} \\ 0, & \text{otherwise} \end{cases} \\
\text{feature}_4(d, q) &= \begin{cases} 1, & \text{if } d \cdot \text{comp} = q \cdot \text{comp} \\ 0, & \text{otherwise} \end{cases} \\
\text{feature}_5(d, q) &= \begin{cases} 1, & \text{if } d \cdot \text{type} = q \cdot \text{type} \\ 0, & \text{otherwise} \end{cases} \\
\text{feature}_6(d, q) &= \frac{1}{1 + |d \cdot \text{prio} - q \cdot \text{prio}|} \\
\text{feature}_7(d, q) &= \frac{1}{1 + |d \cdot \text{vers} - q \cdot \text{vers}|}
\end{aligned} \tag{2}$$

The first two features regard the textual similarity between two bug reports over the fields summary and description. These two textual features are calculated by BM25F_{ext} between bug report d and query bug report q . BM25F [39, 40] is an effective textual similarity function for retrieving documents with structures. The authors of REP extend BM25F by considering term frequencies in queries and proposed BM25F_{ext} .

In feature_1 , summary and description are represented in uni-gram, while in feature_2 , summary and description are represented in bi-gram. Thus, the input of BM25F_{ext} consists of a bag of uni-grams and bi-grams in both features. For feature_{3-5} , they are the categorical features of **product**, **component**, and **type**, respectively. If the corresponding field value from d and q is the same, the value of the feature is 1, otherwise, it is 0. For feature_{6-7} , they are the categorical features of **priority** and **version**, respectively. They are calculated by the reciprocal of the distance between the corresponding field value from d and q . Overall, the REP approach contains 19 free parameters with different initial values. These parameters are tuned by gradient descent.

4 Experimental Design

In this section, we first outline the research questions we aim to investigate. Secondly, we describe the datasets utilized in our study. Thirdly, we discuss the evaluation metrics employed. Fourthly, we provide succinct descriptions of the baseline models. Fifthly, we delve into the six alternatives to ChatGPT considered, with half being open-source LLMs and the other half being statistics-based keyword extraction methods. Lastly, we mention the implementation details for running ChatGPT and its alternatives.

4.1 Research Questions

To understand whether CUPID performs better compared to existing state-of-the-art approaches and whether each component of CUPID is useful, we answer the following two research questions (RQs).

RQ1: *How effective is CUPID compared to the state-of-the-art DBRD approaches?* We compare CUPID with state-of-the-art DBRD techniques, which consider DBRD as a ranking problem, i.e., REP [5], SIAMESE PAIR [6], and SABD [7].

RQ2: *How effective are the components of CUPID?* To answer this RQ, we conduct an ablation study on SPARK dataset to investigate the effectiveness of the components of CUPID. This RQ is further divided into the following sub-RQs:

- **RQ2.1:** *How effective is the selection rule?* Besides content, we also experimented with selecting bug reports whose description is considered to be *long*. We consider bug reports whose description is longer than n words as *long* bug reports. We get n by calculating the 75th percentile of the description length in the training set. The reason why we select long bug reports is that long bug reports are usually not concise and contain long stack traces and code snippets. These long bug reports would make it challenging for REP to retrieve the potential master bug reports. We investigate the impact of the length and content of the bug report description on the performance of CUPID.
- **RQ2.2:** *How effective is ChatGPT compared to other LLMs and statistics-based keyword extraction methods?* We select three open-source LLMs, i.e., LLAMA 3 (i.e., `meta-llama/Meta-Llama-3-8B-Instruct` in Hugging Face library [41]), PHI-3 (i.e., `microsoft/Phi-3-mini-128k-instruct`), and OPENCHAT (i.e., `openchat/openchat-3.5-0106`) based on their performance in MMLU benchmark on the LMSys chatbot Arena leaderboard in May 2024 [42]. All these three LLMs are open-source and have less than 10B parameters. For the statistics-based keyword extraction methods, we select TFIDF [43], KP-MINER [44], and YAKE [45] based on their good performance on a recent survey [46] and popularity in the literature.
- **RQ2.3:** *How effective is the prompt template?* We investigate the impact of the prompt template on the performance of CUPID. To get alternative prompts for ChatGPT, we queried ChatGPT again. The prompt we used to get alternative prompts and the prompts replied by ChatGPT are shown in Table 3.

4.2 Dataset

As mentioned in Section 1, we are concerned about boosting the performance of DBRD, especially in the bug repositories with the typical number of issues. Therefore, the target datasets are those that contain a typical number of issues. We employ **three datasets**, i.e., Spark, Hadoop, and Kibana datasets, which are provided by a recent benchmarking study by Zhang et al. [16]. These datasets contain around 10k issues each, which is considered a typical number of issues. These datasets are recent issues, ranging from 2018 to 2022, which addressed the **age** bias, i.e., the model performs differently on the recent data and old data. Spark and Hadoop are two popular open-source distributed computing frameworks. They both use **Jira** as their issue tracking

Table 3: Query to ChatGPT to Generate Alternative Prompt Templates and ChatGPT’s Responses.

To Get Alternative Prompt Templates

Provide two alternative prompts: one is more concise, and the other is more verbose. The current prompt template is:

Identify keywords from the summary and description of the bug report that can be used to detect duplicates.

Output format:

Summary: [Selected Keywords]

Description: [Selected Keywords]

More Concise Prompt Template

Identify keywords from the bug report to detect duplicates.

Output format:

Summary: [Selected Keywords]

Description: [Selected Keywords]

Summary: {{Summary}}

Description: {{Summary}}

More Verbose Prompt Template

Review the summary and description of the bug report to identify specific keywords that can be used as criteria for detecting duplicate reports. Consider the language used, technical terms, and any unique identifiers mentioned in the report.

Output format:

Summary: [Selected Keywords]

Description: [Selected Keywords]

Summary: {{Summary}}

Description: {{Description}}

Table 4: Dataset statistics.

Dataset	# BRs	# Pairs		Test	
		Train	Valid	Total BRs	Dup. BRs
Spark	9,579	625	26	2,841	81
Hadoop	14,016	626	26	3,740	92
Kibana	17,016	724	28	7,167	184

system. Kibana is a visualization tool for Elasticsearch, and it uses [GitHub](#) as its issue tracking system. The statistics of the datasets are shown in Table 4. The duplicate and non-duplicate pairs were sampled by Zhang et al. [16]. Their ratio is 1:1. We obtained the data in the dataset provided by Zhang et al. In our experiment, we fixed the number of training and validation pairs. The number of duplicate bug reports in the test set is the bug reports we investigate. We report the performance of each approach in terms of how they perform in retrieving the master bug reports for the duplicate bug reports in the test set.

4.3 Evaluation Metrics

Following prior works on DBRD [6, 16, 17, 47], we only use Recall Rate@ k (RR@ k) as the evaluation metric, where k represents the number of bug reports to be considered. Note that a few other works have also adopted Mean Average Precision (MAP) in the DBRD literature. However, since MAP considers all of the predicted positions, it is not suitable for our case, where only the top k predictions matter. This is based on real-world practice, where developers are more likely to check the top k predictions rather than all of the predictions. A survey on practitioners’ expectations towards fault localization also shows that around 98% of respondents are not willing to check the predictions beyond the top-10 to find the faulty element [48]. Additionally, the DBRD feature adopted in the microsoft/vscode repository also only shows at most top-5 predictions [16].

Furthermore, as already discussed in early work [17], the two widely used metrics in information retrieval, i.e., Precision@ k and Recall@ k , do not fit into how DBRD works. For each query bug report, we only have a master bug report to look for (i.e., the relevant item is only 1). Consider $k = 10$, a successful prediction would lead to Precision@10=1/10(10%), and Recall@10=1/1(100%); a failed prediction would lead to Precision@10=0 and Recall@10=0. Therefore, we adopt RR@ k as the evaluation metric.

Following the definition in prior works [5, 7, 29, 49], RR@ k is defined as the percentage of duplicate bug reports that are correctly assigned to the bucket they belong to when a model makes a top- k prediction for each test bug report. In our experiment, RR@ k will measure how well DBRD techniques correctly link the duplicate bug reports to their master bug report. A higher RR@ k indicates that more bug reports in the test set are correctly linked to the bucket they belong to when a model retrieves top- k prediction.

$$\text{Recall Rate} = \frac{N_{\text{recalled}}}{N_{\text{total}}} \quad (3)$$

Formula 3 shows how to calculate the Recall Rate. N_{recalled} refers to the number of duplicate bug reports whose bucket (master bug report) are in the suggested list (with a size of $[1, 2, \dots, k]$). N_{total} refers to the number of duplicate bug reports investigated. Considering different sizes of the suggested list, i.e., k , we can get RR@ k . Following the benchmarking work by Zhang et al. [16], in our case, we consider at most 10 predictions, i.e., $k = [1, 2, \dots, 10]$.

4.4 DBRD Baselines

REP [5] The details of REP can be referred in Section 3.3.

Siamese Pair [6] is the first approach that leverages deep learning for DBRD. As its name suggests, SIAMESE PAIR utilizes Siamese variants of Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) trained on max-margin objective to distinguish similar bugs from non-similar bugs. SIAMESE PAIR adopts word embedding to represent the textual data as numerical vectors. Then, it employs

three different types of neural networks to encode **summary**, **description**, and categorical features according to their properties. Specifically, **summary** is encoded by bi-LSTM, **description** is encoded by a CNN, while the categorical information is encoded with a single-layer neural network. When a new bug report comes, SIAMESE PAIR encodes the bug report with the trained model. It then calculates the cosine similarity between the new bug report and each bug report in the master set and gives top k predictions.

SABD [7] is the latest deep learning-based DBRD approach. It comprises two sub-network modules, each comparing textual and categorical data from two bug reports. In the textual sub-network, the soft-attention alignment mechanism [50] compares each word in a bug report with a fixed-length representation of all words in the other bug report. By doing so, SABD learns the joint representation of bug reports. In the categorical sub-network, each categorical field relates to a lookup table that links the field value to a real-valued vector. The output vector from each sub-module is concatenated and fed to a fully connected layer. Finally, a classifier layer, which is a logistic regression, produces the final prediction, i.e., whether the two bug reports are duplicates.

4.5 ChatGPT Alternatives

4.5.1 Open-source LLMs

Llama 3 [20] is the latest LLMs developed and released by Meta. LLAMA 3 is an autoregressive LLM that uses an optimized Transformer architecture. The tuned versions use supervised fine-tuning and reinforcement learning with human feedback to align with human preferences. It comes in two sizes, i.e., 8B and 70B versions, both of which use grouped-query attention for improved inference scalability. The LLAMA 3 instruction-tuned models are optimized for dialogue use cases and outperform many of the available open-source chat models on common industry benchmarks.

Phi-3 [22] is a family of open-source LLMs developed by Microsoft. According to Microsoft, the PHI-3-MINI model, with “only” 3.8 billion parameters, outperforms models twice its size, while PHI-3-SMALL and PHI-3-MEDIUM surpass the performance of much larger models. PHI-3-MINI is available in two context-length variants: 4K and 128K tokens. Notably, it is the first model in its class to support a context window of up to 128K tokens, with minimal impact on quality. PHI-3-MINI are instruction-tuned and trained to follow different types of instructions.

OpenChat[23] are trained on general supervised fine-tuning data, consisting of a small amount of expert data mixed with a large proportion of sub-optimal data, without any preference labels. Despite this, OPENCHAT models deliver exceptional performance on par with ChatGPT, even with a 7B model that can be run on consumer-grade GPUs (e.g., RTX 3090). OPENCHAT introduces the C(onditioned)-RLFT technique, which regards different data sources as coarse-grained reward labels and learns a class-conditioned policy to leverage complementary data quality information. Upon its release in January 2024, **OpenChat-3.5-0106** was the strongest 7B language model in the world, ranking as the top 7B model on the LMSys Arena [42], and the top open-source language model not originating from a company.

4.5.2 Statistics-based Keyword Extraction Methods

TF-IDF [43] serves as the common baseline for the keyword extraction task. TF-IDF scores and ranks phrases according to the formula: $\text{TF-IDF} = \text{Tf} \times \text{Idf}$, where Tf is the raw phrase frequency, and $\text{Idf} = \log_2 \frac{N}{1 + |\{d \in D : \text{phrase} \in d\}|}$. In the Idf formula, N represents the number of documents in the document set D , and $|\{d \in D : \text{phrase} \in d\}|$ denotes the number of documents in which the phrase appears.

KP-Miner [44] not only utilizes term frequency and inverse document frequency scores, but also incorporates various other statistical measures. It employs an efficient candidate phrase filtering process and uses a scoring function. Specifically, the system retains candidate words that are not separated by punctuation or stop words, considering the minimum allowed visible frequency factor and a cutoff constant defined based on the number of words after the first occurrence of a phrase. The system then ranks candidate phrases based on their Tf and Idf scores, as well as the term position and the lift factor of the compound term relative to its individual terms.

YAKE [45] utilizes novel statistical metrics to capture contextual information and the spread of terms within the document, in addition to considering the position and frequency of terms. First, YAKE preprocesses the text by splitting it into separate terms. Second, a set of five features is calculated for each individual term: casing, word position, word frequency, the number of different terms appearing on the left and right sides of the word, and how often a candidate word appears within different sentences. All these features are then utilized to calculate the $S(w)$ score for each term. Finally, a 3-gram sliding window generates a continuous sequence of 1-gram, 2-gram, and 3-gram candidate keywords.

4.6 Implementation

ChatGPT. Given that ChatGPT is still fast evolving, it has undergone several iterations [51]. In this study, we worked on the GPT-3.5 Turbo version. To interact with ChatGPT, we used its official API. Although GPT-4 is also available, it is much more expensive than GPT-3.5. Therefore, we chose to use the cheaper version of ChatGPT, which we believe to have a wider range of users compared to GPT-4. As such, our results would be more valuable as they apply to a wider range of users.

Since ChatGPT may generate different answers for the same query, we set the temperature to 0 and fixed the random seed to 42 to make the answers more stable. Even so, we still observed some variations in the answers generated by ChatGPT. Thus, we ran ChatGPT five times for each bug report and ran the REP five times to get the average results. Specifically, we average the $\text{RR}@k$ of the five runs of CUPID.

Open-source LLMs. We used the Hugging Face Transformers library [41] to interact with the open-source LLMs. Since we set `do_sample=False`, the answers generated by the open-source LLMs are stable and do not change across runs. The LLM models are run only once for each bug report. We set the `max_new_tokens=2,048`, which means the maximum number of tokens to generate is 2,048. We use the same prompt templates across different LLMs.

Table 5: Recall Rate@ k obtained on the Spark, Hadoop, and Kibana datasets.

	RR@1	RR@2	RR@3	RR@4	RR@5	RR@10
<i>Spark</i>						
REP	0.346	0.383	0.457	0.481	0.481	0.556
SIAMESE PAIR	0.037	0.049	0.059	0.064	0.074	0.121
SABD	0.202	0.247	0.281	0.294	0.304	0.331
CUPID	0.360	0.435	0.472	0.499	0.519	0.602
<i>Hadoop</i>						
REP	0.402	0.489	0.522	0.554	0.576	0.609
SIAMESE PAIR	0.033	0.046	0.057	0.063	0.076	0.093
SABD	0.215	0.267	0.293	0.304	0.324	0.411
CUPID	0.420	0.496	0.522	0.546	0.587	0.637
<i>Kibana</i>						
REP	0.364	0.440	0.527	0.560	0.587	0.620
SIAMESE PAIR	0.020	0.036	0.050	0.063	0.076	0.092
SABD	0.293	0.382	0.428	0.467	0.489	0.555
CUPID	0.389	0.490	0.557	0.580	0.605	0.654

Statistics-based keyword extraction methods. We used the open-source keyword extraction toolkit [52] to run the methods. Since TF-IDF, KP-MINER, and YAKE are deterministic, the keywords extracted by these methods are stable and do not change across runs. Thus, we run them once for each bug report. As TF-IDF and KP-MINER rely on the document frequency of the terms, we used the training set to calculate the document frequency of the terms. We leave the hyper-parameters of these keyword extraction methods as default.

5 Results

5.1 RQ1: Comparing with baselines

Table 5 shows the results of CUPID and the baselines on the Spark, Hadoop, and Kibana datasets. Overall, CUPID consistently improves the DBRD performance in terms of RR@10 on all three datasets, yielding an improvement of 8% (Spark), 5% (Hadoop), and 6% (Kibana) over the prior state-of-the-art approach REP. This improvement is obtained by successfully utilizing the content generation ability of ChatGPT to transform the bug reports into a format where only essential information is kept. Compared with the best-performing deep learning-based approach, i.e., SABD, we observe an improvement of **up to 82%** on the Spark dataset. In the low-volume datasets, SABD and SIAMESE PAIR lose to non-deep learning approaches, i.e., REP and CUPID.

Comparing the performance of SIAMESE PAIR and SABD in all three datasets, we can find that SIAMESE PAIR suffers more from the challenge of limited training data. SIAMESE PAIR performs less than 50% of SABD in all three datasets in terms

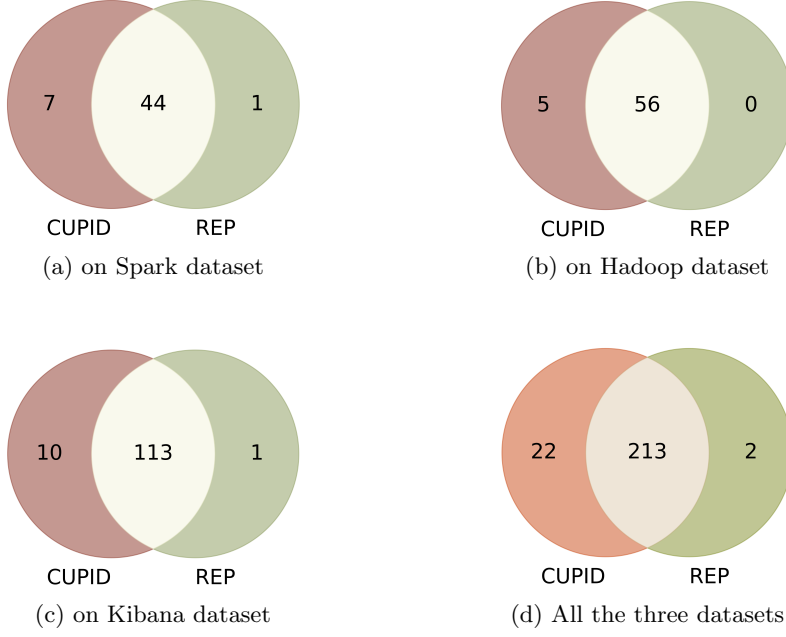


Fig. 2: Successful prediction Venn diagram

of RR@10. We argue that when there is a lack of adequate training data, it is less meaningful to compare different deep learning-based models.

Dataset-wise, all approaches perform relatively worse on the Spark dataset and relatively better on the Kibana dataset. The observation aligns with the findings from prior studies [7]: the same DBRD approach, i.e., SABD, achieves a variety of RR@10 on different datasets examined, ranging from 0.55 (on OpenOffice dataset) to 0.7 (on Netbeans dataset). The results demonstrate that the performance of a DBRD technique is dependent on the characteristics of the dataset being used. This observation suggests that it would be advantageous to tune the prompt template based on the unique characteristics of each dataset, such as incorporating dataset-specific stopwords into the prompts to prevent the LLMs from listing them as keywords. Tailoring the prompt templates to the nuances of individual datasets can potentially enhance the effectiveness of the DBRD technique and improve its overall performance. We leave this for future work to boost the performance further.

Figure 2 shows the Venn diagrams for successful predictions made by the prior state-of-the-art method, i.e., REP, and CUPID on each dataset and all datasets combined. We see that CUPID successfully retrieves more master bug reports compared to REP. On the Hadoop dataset, only CUPID successfully retrieved more master bug reports, while REP did not successfully retrieve more.

To demonstrate the ability of CUPID, we show an example, i.e., the query bug report is HADOOP-17091[53] where REP failed to predict the correct master bug report in the top-10 positions, while CUPID managed to. Figure 3 shows the



[JDK11] Fix Javadoc errors

Description

```
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 17.982 s
[INFO] Finished at: 2020-06-20T01:56:28Z
[INFO] -----
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-javadoc-plu
[ERROR] Exit code: 1 - javadoc: warning - You have specified the HTML ver
[ERROR] The default is currently HTML5 and the support for HTML 4.01 will
[ERROR] in a future release. To suppress this warning, please ensure that
[ERROR] in your comments are valid in HTML5, and remove the -html4 option
[ERROR] /home/jenkins/jenkins-slave/workspace/hadoop-multibranch_PR-2084/
[ERROR] com.google.protobuf.GeneratedMessageV3 implements
[ERROR] ^
[ERROR] symbol: class GeneratedMessageV3
[ERROR] location: package com.google.protobuf
[ERROR] /home/jenkins/jenkins-slave/workspace/hadoop-multibranch_PR-2084/
[ERROR] com.google.protobuf.GeneratedMessageV3 implements
[ERROR] ^
[ERROR] symbol: class GeneratedMessageV3
[ERROR] location: package com.google.protobuf
[ERROR] /home/jenkins/jenkins-slave/workspace/hadoop-multibranch_PR-2084/
```

Fig. 3: The case where CUPID succeeded while REP failed: HADOOP-17091

summary and description of this issue. We can see that there is no natural language in the description, containing only error messages. Thus, REP considered the most possible master bug report to be HADOOP-16648 [54], which also contains a large portion of the error messages. We checked the single-run result by ChatGPT. Thanks to the language understanding and generation ability of ChatGPT, CUPID identified the keywords: Maven, BUILD FAILURE, Javadoc, error, HTML version, HTML5, HTML4, warning, symbol, class, GeneratedMessageV3, package, com.google.protobuf from the description of HADOOP-17091. The generated shorter description on the query bug report has several words that overlap with the description of the real master bug report (HADOOP-16862 [55]). It enables CUPID to successfully rank it at the top 10 position. Since the real master bug report has a long error message as the description, REP failed to retrieve it. This example shows that ChatGPT can be helpful when long descriptions contain a build failure message. It can generate the most important keywords, which are vital for duplicate detection.

Error Analysis. We also investigate the two cases when CUPID failed to predict the correct master bug report in the top-10 positions, while REP managed to do so. One sample is SPARK-31519[56]. If we check the keywords

Table 6: Number of test bugs and bugs that need to run after adopting selection rules.

Selection Rule	Dataset		
	Spark	Hadoop	Kibana
None	2,841	3,740	7,167
w/ Length	586	903	1,883
w/ Content	1,466	1,537	3,048
w/ Length + Content	1,583	1,845	3,791

extracted by ChatGPT solely, which are Spark, SQL parser, Filter operator, Aggregate operator, ResolveAggregateFunctions, ResolveReferences, ResolveFunctions, ResolveTimeZone, Analyzer rule, Grouping columns, we can see that the keywords are reasonable. However, since the description shrinks the test history, which contains the command and output, CUPID failed to retrieve the correct master bug report SPARK-31334[57], as it also contains the test history similar to the query bug report. The other sample is issue #66249[58] in Kibana. The query bug report contains the Kibana issue template. The keywords extracted by ChatGPT include the template keywords, which are useless for duplicate detection. Apart from the template keywords, ChatGPT considers FireFox as one of the keywords. This leads to our method retrieving the wrong master bug report #43282[59] at the first position, which is a bug report related to FireFox. These examples demonstrate that, in some cases, keywords alone are not sufficient to identify duplicate bug reports accurately. To address such cases, we can consider using a prompt template that guides ChatGPT in retaining concise code and output history while ignoring the template keywords.

Answer to RQ1: CUPID outperforms the best baseline by 8%, 5%, and 6% in terms of Recall Rate@10 on the Spark, Hadoop, and Kibana datasets, respectively.

5.2 RQ2: Ablation Study

5.2.1 RQ2.1: Comparison of Different Selection Rules

Table 6 shows how many bug reports need to query ChatGPT after adopting (1) no-selection rules, (2) selection by length, (3) selection by content, and (4) selection by both length and content. If we only use *length* as the selection criteria, we will only need to run ChatGPT on 21%, 24%, and 26% of the original test bug reports in Spark, Hadoop, and Kibana datasets, respectively. While the computational cost would be reduced, it is essentially a trade-off: we want to achieve both efficiency and accuracy, which can be contradictory in some cases. We want to take full advantage of ChatGPT with minimal computational costs. Other than length, we also identify the *content* criteria. If we use *content* as the only selection criteria, we will need to run ChatGPT on 52%, 41%, and 43% of the original test bug reports in Spark, Hadoop, and Kibana datasets, respectively. After adopting both length and content criteria, the bug reports

Table 7: Ablation study on selection rules: Recall Rate@ k obtained on the Spark dataset.

Selection	RR@1	RR@2	RR@3	RR@4	RR@5	RR@10
None	0.365	0.407	0.464	0.489	0.519	0.575
w/ Length	0.336	0.398	0.422	0.474	0.481	0.568
w/ Content	0.360	0.435	0.472	0.499	0.519	0.602
w/ Length + Content	0.348	0.422	0.469	0.494	0.516	0.602

Table 8: Ablation study on ChatGPT: Recall Rate@ k obtained on the Spark dataset.

Model	RR@1	RR@2	RR@3	RR@4	RR@5	RR@10
LLAMA 3	0.321	0.432	0.444	0.469	0.506	0.605
PHI-3	0.309	0.383	0.432	0.481	0.494	0.519
OPENCHAT	0.321	0.358	0.383	0.432	0.457	0.531
TFIDF	0.383	0.457	0.494	0.494	0.519	0.556
KP-MINER	0.358	0.444	0.469	0.481	0.481	0.531
YAKE	0.321	0.407	0.481	0.481	0.494	0.556
CUPID	0.360	0.435	0.472	0.499	0.519	0.602

Table 9: Average number of keywords extracted by different methods.

Model	Average Keywords	
	Summary	Description
LLAMA 3	5.1	10.9
PHI-3	15.7	36.3
OPENCHAT	10.7	21
TFIDF	11.3	11.6
KP-MINER	11.0	13.9
YAKE	11.7	15.2
CUPID	5.3	18.0

needed to be processed by ChatGPT increased and accounted for 56%, 49%, and 53%, which still saved more than 40% bug reports from processing.

Table 7 shows the corresponding results of applying selection rules. Comparing the performance of no selection rules, i.e., querying all the test bug reports with ChatGPT and applying *both* selection rules, we can observe that after applying the rules, RR@10 improves by 8%. We also observe that only using the content as the selection rule can achieve the same performance as using both selection rules; we can conclude that the content is the most important selection rule. Other than making an improvement on the accuracy, it also frees at least 40% of the bug reports in the test set from querying ChatGPT. Here, we do not only save the computational cost but also improve accuracy. Thus, we use the content as the selection rule in CUPID.

Table 10: Ablation study on alternative prompt templates: Recall Rate@ k obtained on the Spark dataset.

Method	RR@1	RR@2	RR@3	RR@4	RR@5	RR@10
Concise	0.346	0.415	0.479	0.501	0.526	0.595
Verbose	0.328	0.370	0.395	0.432	0.452	0.499
CUPID	0.360	0.435	0.472	0.499	0.519	0.602

5.2.2 RQ2.2: Comparison of Different Keyword Extractors

Compared with ChatGPT, LLAMA 3 achieves comparable performance in terms of RR@10, while PHI-3 and OPENCHAT perform much worse. The excellent performance of LLAMA 3 makes it promising to use an open-source LLM as a substitute for ChatGPT for the DBRD task. Further investigation on when and why open-source LLMs lose to ChatGPT can be put to take full advantage of the latest advancement of LLMs. In comparison with the statistics-based keyword extraction methods, ChatGPT outperforms TFIDF by 8%, KP-MINER by 13%, and YAKE by 8% in terms of RR@10.

Table 9 shows the average number of keywords extracted by different methods. We can observe that LLAMA 3 extracts the least number of keywords, while PHI-3 and OPENCHAT extract the most. To understand why PHI-3 and OPENCHAT extract more keywords, we examined the generated keywords. We found that PHI-3 generated description keywords over 1,000 words on 9 occasions. In such cases, it tended to repeat certain words until reaching the output token limit. For instance, the description keywords generated by PHI-3 for the bug report SPARK-30412 [60] contained the phrase `deprecated methods in Java test suites` 246 times. This scenario was observed 3 times in OPENCHAT-, once in LLAMA 3-, and 7 times in ChatGPT-generated keywords (note that ChatGPT has been run 4 times more than the other LLMs). As it is generally expected that keywords should not be repeatable and can be easily made unique, the low ratio of such cases in our experiment does not seem to be a significant issue. In the future, we can mitigate this generation issue by keeping only the unique set of the extracted keywords. For the statistics-based keyword extraction methods, we used the default parameters, with the relevant parameter set to 10, i.e., `get_n_best=10`, retrieving the 10 highest-scored candidates as keywords or phrases.

In terms of efficiency, on average, all the experimented methods took less than 5 seconds to make a prediction. There is no significant difference in terms of efficiency among the methods. Note that the actual time LLM takes for each specific bug report is different, as it depends on the length and complexity of the bug report.

5.2.3 RQ2.3: Comparison of Different Prompt Templates

While running ChatGPT with different prompts, we found sometimes they cannot follow the desired output format. If ChatGPT’s response did not provide separate keywords for the summary and description, instead returning a mixed list of keywords, we would repeat the query up to 5 times. If the response continued to return only a list of keywords without distinguishing between summary and description, we would

extract these keywords and use them as both the summary and description. For all 7,330 queries (i.e., 5 runs of 1,466 test samples in the Spark dataset): (1) using the prompt template from CUPID, ChatGPT successfully provided separate keywords for the summary and description; (2) with the verbose prompt template, ChatGPT produced a mixed list of keywords 1,239 times; (3) with the concise prompt template, ChatGPT produced a mixed list only 39 times. This indicates that when using the more verbose prompt template, ChatGPT tends not to maintain the desired output format.

Table 10 shows the results of querying ChatGPT with the two alternative templates and the template employed in CUPID. We observe that the verbose prompt, which is more comprehensive than the concise prompt, leads to worse performance, being 17% worse than the final prompt template in terms of RR@10. The concise and final templates convey very similar meanings and do not have a significant impact on performance. These results indicate the significance of prompt engineering.

Answer to RQ2: Selection rules effectively improve the performance of CUPID and reduce the computational cost. Furthermore, LLAMA 3 can be a promising open-source LLM alternative for extracting keywords. Suitable prompt templates are crucial for the performance of ChatGPT.

6 Threats to Validity

Threats to Internal Validity. The main internal threat is whether there is data leakage in ChatGPT and the open-source LLMs. However, since we do not have access to the training data of LLMs, we cannot verify whether there is data leakage in LLMs. Even so, since we did not directly use LLMs to compare whether two bug reports are duplicates, instead, we utilize LLMs indirectly, which may not benefit much from memorizing the training data. Thus, we believe this threat is minimal.

Threats to External Validity. The primary external threat is the generalizability of our findings. Our focus in this study is on datasets with a typical number of bug reports, roughly 10k issues. Therefore, our results may not extend to datasets with a significantly greater number of bug reports, such as those containing tens of thousands of issues. Nevertheless, we believe that our findings remain valuable for the majority of projects. This is supported by the fact that in a dataset of 994 high-quality projects from [GitHub](#), each project contains an average of 2k issues [11].

7 Related Work

Except for the works that studied the DBRD task, other automated bug report management tasks have also attracted much research interest [61–64]. In this section, we briefly introduce several studies on automated bug report management tasks, including bug component assignment [62], developer assignment [63], and issue title generation [65]. Su et al. [62] propose a learning-to-rank framework that leverages the correct bug assignment history to assign the most appropriate component to a bug. Instead of only using the features from bug reports, their approach also derives rich features from

this knowledge graph. Lee et al. [63] focus on assigning a bug report to the most appropriate developer. They propose a framework (i.e., LBT-P) that applies LLMs, such as RoBERTa [66], to extract semantic information. LBT-P uses knowledge distillation to compress an LLM into a small and fast model. Additionally, it also introduces knowledge preservation fine-tuning to handle the challenge of catastrophic forgetting [67] of LLM. Although the issue is not a new concept, the automatic issue title generation task has not been studied until recently [65, 68]. Zhang et al. [65] propose to leverage the state-of-the-art BART [69] model to generate issue titles for bug reports. The experimental results show that fine-tuning BART can better generate issue titles than the prior state-of-the-art approach, i.e., iTAPE [68], based on sequence-to-sequence model [70].

8 Conclusion and Future Work

In this work, we focus on the task of DBRD, especially on projects with a typical number of bug reports. We investigated how to combine the advantages of both the traditional DBRD approach and LLMs and proposed CUPID. CUPID leverages ChatGPT to identify keywords as the input of the state-of-the-art traditional DBRD approach REP. We conduct a comprehensive evaluation on three datasets and compare CUPID with three baselines. We also compare ChatGPT with its alternatives, i.e., three open-source LLMs, and three statistical-based keyword extraction techniques. The experimental results show that CUPID outperforms the state-of-the-art DBRD techniques regarding **Recall Rate@10** on all the datasets. Particularly, CUPID achieves high **Recall Rate@10** scores ranging from **0.602 to 0.654** on all the datasets investigated.

We envision deploying CUPID in practical settings without introducing significant overhead. The query time for a single bug report using ChatGPT is remarkably efficient, averaging less than 5 seconds. This rapid turnaround time attests to the efficiency of the underlying LLMs and the optimized implementation. It ensures that the integration of CUPID into existing bug-triaging workflows is seamless. In the future, we plan to package CUPID as a tool for practitioners. Additionally, we are interested in exploring the application of ChatGPT to other bug report management tasks, such as bug component and developer assignment, leveraging its natural language understanding capabilities.

Availability. Our replication package is publically available at <https://github.com/soarsmu/CUPID>.

References

- [1] Bugzilla. <https://www.bugzilla.org/>. (Accessed on 05/05/2024) (2024)
- [2] Jira — Issue & Project Tracking Software — Atlassian. <https://www.atlassian.com/software/jira>. (Accessed on 05/05/2024) (2024)
- [3] GitHub. <https://github.com/>. (Accessed on 05/05/2024) (2024)

- [4] Lazar, A., Ritchey, S., Sharif, B.: Generating duplicate bug datasets. In: Proceedings of the 11th Working Conference on Mining Software Repositories, pp. 392–395 (2014)
- [5] Sun, C., Lo, D., Khoo, S.-C., Jiang, J.: Towards more accurate retrieval of duplicate bug reports. In: 2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011), pp. 253–262 (2011). IEEE
- [6] Deshmukh, J., Annervaz, K., Podder, S., Sengupta, S., Dubash, N.: Towards accurate duplicate bug retrieval using deep learning techniques. In: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 115–124 (2017). IEEE
- [7] Rodrigues, I.M., Aloise, D., Fernandes, E.R., Dagenais, M.: A soft alignment model for bug deduplication. In: Proceedings of the 17th International Conference on Mining Software Repositories, pp. 43–53 (2020)
- [8] He, J., Xu, L., Yan, M., Xia, X., Lei, Y.: Duplicate bug report detection using dual-channel convolutional neural networks. In: Proceedings of the 28th International Conference on Program Comprehension, pp. 117–127 (2020)
- [9] Xiao, G., Du, X., Sui, Y., Yue, T.: Hindbr: Heterogeneous information network based duplicate bug report prediction. In: 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE), pp. 195–206 (2020). IEEE
- [10] Riazi, M.S., Rouani, B.D., Koushanfar, F.: Deep learning on private data. IEEE Security & Privacy **17**(6), 54–63 (2019)
- [11] Joshi, S.D., Chimalakonda, S.: Rapidrelease-a dataset of projects and issues on github with rapid releases. In: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), pp. 587–591 (2019). IEEE
- [12] ohmyzsh/ohmyzsh. <https://github.com/ohmyzsh/ohmyzsh>. (Accessed on 05/05/2024) (2023)
- [13] axios/axios: Promise based HTTP client for the browser and node.js. <https://github.com/axios/axios>. (Accessed on 05/05/2024) (2024)
- [14] vuejs/vue: This is the repo for Vue 2. For Vue 3, go to <https://github.com/vuejs/core>. <https://github.com/vuejs/vue>. (Accessed on 05/05/2024) (2024)
- [15] Significant-Gravitas/Auto-GPT. <https://github.com/Significant-Gravitas/Auto-GPT>. (Accessed on 05/05/2024) (2024)
- [16] Zhang, T., Han, D., Vinayakara, V., Irsan, I.C., Xu, B., Thung, F., Lo, D.,

- Jiang, L.: Duplicate bug report detection: How far are we? *ACM Transactions on Software Engineering and Methodology* (2022)
- [17] Runeson, P., Alexandersson, M., Nyholm, O.: Detection of duplicate defect reports using natural language processing. In: 29th International Conference on Software Engineering (ICSE'07), pp. 499–510 (2007). IEEE
 - [18] Jalbert, N., Weimer, W.: Automated duplicate detection for bug tracking systems. In: 2008 IEEE International Conference on Dependable Systems and Networks With FTCS and DCC (DSN), pp. 52–61 (2008). IEEE
 - [19] Sun, C., Lo, D., Wang, X., Jiang, J., Khoo, S.-C.: A discriminative model approach for accurate duplicate bug report retrieval. In: Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 1, pp. 45–54 (2010)
 - [20] AI@Meta: Llama 3 model card (2024)
 - [21] Introducing Phi-3: Redefining what's possible with SLMs — Microsoft Azure Blog. <https://azure.microsoft.com/en-us/blog/introducing-phi-3-redefining-whats-possible-with-slms/>. (Accessed on 05/13/2024) (2024)
 - [22] Abdin, M., Jacobs, S.A., Awan, A.A., Aneja, J., Awadallah, A., Awadalla, H., Bach, N., Bahree, A., Bakhtiari, A., Behl, H., et al.: Phi-3 technical report: A highly capable language model locally on your phone. arXiv preprint arXiv:2404.14219 (2024)
 - [23] Wang, G., Cheng, S., Zhan, X., Li, X., Song, S., Liu, Y.: Openchat: Advancing open-source language models with mixed-quality data. In: The Twelfth International Conference on Learning Representations (2024)
 - [24] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of NAACL-HLT, pp. 4171–4186 (2019)
 - [25] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al.: Language models are few-shot learners. *Advances in neural information processing systems* **33**, 1877–1901 (2020)
 - [26] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al.: Language models are unsupervised multitask learners. *OpenAI blog* **1**(8), 9 (2019)
 - [27] Introducing ChatGPT. <https://openai.com/blog/chatgpt>. (Accessed on 05/05/2024) (2024)

- [28] Bettenburg, N., Just, S., Schröter, A., Weiss, C., Premraj, R., Zimmermann, T.: What makes a good bug report? In: Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 308–318 (2008)
- [29] Wang, X., Zhang, L., Xie, T., Anvik, J., Sun, J.: An approach to detecting duplicate bug reports using natural language and execution information. In: Proceedings of the 30th International Conference on Software Engineering, pp. 461–470 (2008)
- [30] Nguyen, A.T., Nguyen, T.T., Nguyen, T.N., Lo, D., Sun, C.: Duplicate bug report detection with a combination of information retrieval and topic modeling. In: Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering, pp. 70–79 (2012)
- [31] Sureka, A., Jalote, P.: Detecting duplicate bug report using character n-gram-based features. In: 2010 Asia Pacific Software Engineering Conference, pp. 366–374 (2010). IEEE
- [32] Pennington, J., Socher, R., Manning, C.D.: Glove: Global vectors for word representation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 1532–1543 (2014)
- [33] Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781 (2013)
- [34] Fu, T.-y., Lee, W.-C., Lei, Z.: Hin2vec: Explore meta-paths in heterogeneous information networks for representation learning. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, pp. 1797–1806 (2017)
- [35] Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G.: Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys* **55**(9), 1–35 (2023)
- [36] Bang, Y., Cahyawijaya, S., Lee, N., Dai, W., Su, D., Wilie, B., Lovenia, H., Ji, Z., Yu, T., Chung, W., et al.: A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity. arXiv preprint arXiv:2302.04023 (2023)
- [37] Zhang, T., Irsan, I.C., Thung, F., Lo, D.: Revisiting sentiment analysis for software engineering in the era of large language models. arXiv preprint arXiv:2310.11113 (2023)
- [38] Zhang, W., Deng, Y., Liu, B., Pan, S.J., Bing, L.: Sentiment analysis in the era of large language models: A reality check. arXiv preprint arXiv:2305.15005 (2023)

- [39] Robertson, S., Zaragoza, H., Taylor, M.: Simple bm25 extension to multiple weighted fields. In: Proceedings of the Thirteenth ACM International Conference on Information and Knowledge Management, pp. 42–49 (2004)
- [40] Zaragoza, H., Craswell, N., Taylor, M.J., Saria, S., Robertson, S.E.: Microsoft cambridge at trec 13: Web and hard tracks. In: Trec, vol. 4, pp. 1–1 (2004)
- [41] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T.L., Gugger, S., Drame, M., Lhoest, Q., Rush, A.M.: Transformers: State-of-the-art natural language processing. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 38–45. Association for Computational Linguistics, Online (2020). <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [42] LMSys Chatbot Arena Leaderboard - a Hugging Face Space by lmsys. <https://huggingface.co/spaces/lmsys/chatbot-arena-leaderboard>. (Accessed on 05/05/2024) (2024)
- [43] Rose, S., Engel, D., Cramer, N., Cowley, W.: Automatic keyword extraction from individual documents. Text mining: applications and theory, 1–20 (2010)
- [44] El-Beltagy, S.R., Rafea, A.: Kp-miner: A keyphrase extraction system for english and arabic documents. Information systems **34**(1), 132–144 (2009)
- [45] Campos, R., Mangaravite, V., Pasquali, A., Jorge, A., Nunes, C., Jatowt, A.: Yake! keyword extraction from single documents using multiple local features. Information Sciences **509**, 257–289 (2020)
- [46] Papagiannopoulou, E., Tsoumakas, G.: A review of keyphrase extraction. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery **10**(2), 1339 (2020)
- [47] Amoui, M., Kaushik, N., Al-Dabbagh, A., Tahvildari, L., Li, S., Liu, W.: Search-based duplicate defect detection: An industrial experience. In: 2013 10th Working Conference on Mining Software Repositories (MSR), pp. 173–182 (2013). IEEE
- [48] Kochhar, P.S., Xia, X., Lo, D., Li, S.: Practitioners’ expectations on automated fault localization. In: Proceedings of the 25th International Symposium on Software Testing and Analysis, pp. 165–176 (2016)
- [49] Schütze, H., Manning, C.D., Raghavan, P.: Introduction to Information Retrieval vol. 39. Cambridge University Press Cambridge, ??? (2008)
- [50] Parikh, A., Täckström, O., Das, D., Uszkoreit, J.: A decomposable attention model for natural language inference. In: Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, pp. 2249–2255 (2016)

- [51] ChatGPT — Release Notes — OpenAI Help Center. <https://help.openai.com/en/articles/6825453-chatgpt-release-notes>. (Accessed on 05/05/2024) (2023)
- [52] Boudin, F.: pke: an open source python-based keyphrase extraction toolkit. In: Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: System Demonstrations, Osaka, Japan, pp. 69–73 (2016). <http://aclweb.org/anthology/C16-2015>
- [53] [HADOOP-17091] [JDK11] Fix Javadoc errors - ASF JIRA. <https://issues.apache.org/jira/browse/HADOOP-17091>. (Accessed on 05/05/2024) (2024)
- [54] [HADOOP-16648] HDFS Native Client does not build correctly - ASF JIRA. <https://issues.apache.org/jira/browse/HADOOP-16648>. (Accessed on 05/05/2024) (2024)
- [55] [HADOOP-16862] [JDK11] Support JavaDoc - ASF JIRA. <https://issues.apache.org/jira/browse/HADOOP-16862>. (Accessed on 05/05/2024) (2024)
- [56] [SPARK-31519] Cast in having aggregate expressions returns the wrong result. <https://issues.apache.org/jira/browse/SPARK-31519>. (Accessed on 05/05/2024) (2024)
- [57] [SPARK-31334] Use agg column in Having clause behave different with column type. <https://issues.apache.org/jira/browse/SPARK-31334>. (Accessed on 05/05/2024) (2024)
- [58] [Discover] "Reset search" is broken · Issue #66249 · elastic/kibana. <https://github.com/elastic/kibana/issues/66249>. (Accessed on 05/05/2024) (2024)
- [59] FireFox Filters Bug · Issue #43282 · elastic/kibana. <https://github.com/elastic/kibana/issues/43282>. (Accessed on 05/05/2024)
- [60] [SPARK-30412] Eliminate warnings in Java tests regarding to deprecated API - ASF JIRA. <https://issues.apache.org/jira/browse/SPARK-30412>. (Accessed on 05/05/2024) (2024)
- [61] Zhou, J., Zhang, H., Lo, D.: Where should the bugs be fixed? more accurate information retrieval-based bug localization based on bug reports. In: 2012 34th International Conference on Software Engineering (ICSE), pp. 14–24 (2012). IEEE
- [62] Su, Y., Xing, Z., Peng, X., Xia, X., Wang, C., Xu, X., Zhu, L.: Reducing bug triaging confusion by learning from mistakes with a bug tossing knowledge graph. In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 191–202 (2021). IEEE
- [63] Lee, J., Han, K., Yu, H.: A light bug triage framework for applying large

- pre-trained language model. In: 37th IEEE/ACM International Conference on Automated Software Engineering, pp. 1–11 (2022)
- [64] Zhang, T., Irsan, I.C., Thung, F., Han, D., Lo, D., Jiang, L.: Automatic pull request title generation. In: 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 71–81 (2022). IEEE Computer Society
 - [65] Zhang, T., Irsan, I.C., Thung, F., Han, D., Lo, D., Jiang, L.: itiger: an automatic issue title generation tool. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 1637–1641 (2022)
 - [66] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692 (2019)
 - [67] McCloskey, M., Cohen, N.J.: Catastrophic interference in connectionist networks: The sequential learning problem. In: Psychology of Learning and Motivation vol. 24, pp. 109–165. Elsevier, ??? (1989)
 - [68] Chen, S., Xie, X., Yin, B., Ji, Y., Chen, L., Xu, B.: Stay professional and efficient: automatically generate titles for your bug reports. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, pp. 385–397 (2020)
 - [69] Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., Zettlemoyer, L.: Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 7871–7880 (2020)
 - [70] Sutskever, I., Vinyals, O., Le, Q.V.: Sequence to sequence learning with neural networks. *Advances in neural information processing systems* **27** (2014)